

操作系统课程设计

基于 RISC-V xv6 的 操作系统机制实现与实验

MIT 6.S081 2020 全部 11 个 Labs

目标等级	A 级 - 完成全部实验内容
实验平台	Ubuntu 22.04/24.04 / RISC-V GCC 11.4/13.3 / QEMU 6.2/8.2
组号	待填写
成员	待填写
日期	2026 年 7 月

阅读导航

正文使用 Word 标题层级，可在导航窗格中直接跳转。项目源码按 Lab 分支组织。

- 项目信息
- 摘要
- 1. 需求分析与总体方案
- 2. Lab 1 : Unix Utilities
- 3. Lab 2 : System Calls
- 4. Lab 3 : Page Tables
- 5. Lab 4 : Traps
- 6. Lab 5 : Lazy Allocation
- 7. Lab 6 : Copy-on-Write Fork
- 8. Lab 7 : Multithreading
- 9. Lab 8 : Locking
- 10. Lab 9 : File System
- 11. Lab 10 : mmap
- 12. Lab 11 : Network Driver
- 13. 自动化测试与结果
- 14. 项目难点总结
- 15. 实验心得
- 16. 参考资料

项目信息

项目	内容
课程	操作系统课程设计
选题	MIT 6.S081 2020 xv6 全部 Labs
目标等级	A 级（完成全部实验内容）
组号	待填写
成员与分工	待填写
实验平台	Ubuntu 22.04/24.04、RISC-V GCC 11.4/13.3、QEMU 6.2/8.2

摘要

本项目以 MIT 6.S081 2020 的 RISC-V xv6 为基础，完成从用户态工具、系统调用、虚拟内存、陷阱处理、惰性分配、写时复制、线程、锁、文件系统、内存映射到网卡驱动的 11 个实验。项目不是单一用户程序，而是持续修改内核的进程、内存、文件、同步和设备模块，并使用课程官方评分器逐项验证。

源码采用一项实验一个 Git 分支的组织方式。这样既保留实验框架给定的不同起点，也能通过 `git diff` 明确观察每项内核修改。`course-project` 分支提供报告、环境安装、全量评分与演示脚本。全部实验在真实 Linux 服务器上交叉编译，在 QEMU 的 RISC-V `virt` 机器上启动并执行测试。

1. 需求分析与总体方案

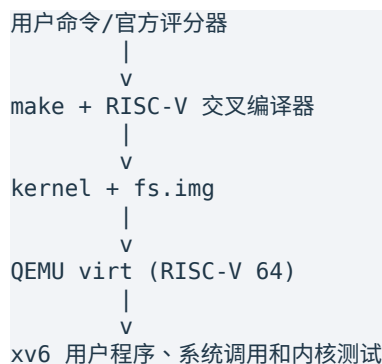
课程要求中，A 级对应“完成一个简单操作系统或全部实验内容”。本项目选择后者，具体完成 MIT xv6 的全部 11 个 Lab，并满足以下验收目标：

1. 每个实验有独立、可追溯的源码分支。
2. 能在 Linux 环境现场编译和启动 xv6。
3. 每个分支能运行其官方 `grade-lab-*` 评分脚本。
4. 报告包含环境、目的、过程、问题、解决方案、结果与心得。
5. 提供自动化脚本，减少答辩时的环境与操作失误。

1.1 仓库组织

course-project	文档、环境脚本、全量评分和演示入口
riscv	原始 RISC-V xv6 基线
util	Lab 1: Unix utilities
syscall	Lab 2: System calls
pgtbl	Lab 3: Page tables
traps	Lab 4: Traps
lazy	Lab 5: Lazy allocation
cow	Lab 6: Copy-on-write
thread	Lab 7: Multithreading
lock	Lab 8: Locking
fs	Lab 9: File system
mmap	Lab 10: mmap
net	Lab 11: Network driver

1.2 运行架构



1.3 环境搭建

项目在两套环境验证：主要评分环境为本机 WSL2 Ubuntu 22.04（16 逻辑核、7.4 GiB 内存、GCC 11.4、QEMU 6.2）；兼容与答辩环境为 Ubuntu 24.04 服务器（2 核、3.8 GiB 内存、GCC 13.3、QEMU 8.2）。安装命令：

```
sudo apt-get update
sudo apt-get install -y \
  qemu-system-misc gcc-riscv64-linux-gnu \
  binutils-riscv64-linux-gnu gdb-multiarch \
  git make python3 python-is-python3
```

仓库已提供等价脚本：

```
sudo ./scripts/setup-ubuntu.sh
```

新版 GCC 会把 xv6 shell 中刻意使用的非返回递归识别为 `-Winfinite-recursion`，而 xv6 又全局启用了 `-Werror`。项目先探测编译器是否支持该警告，再按需使用 `-Wno-error=infinite-recursion`，因此 GCC 11 和 13 都可构建。同时为 `_entry` 增加 `.globl`，保证新版 binutils 能明确识别内核入口。`net` 分支还合入官方 PMP 初始化修复，使 QEMU 6 及以上版本能从 M-mode 正常进入 S-mode。

2. Lab 1 : Unix Utilities

2.1 实验目的

熟悉 xv6 用户态接口、进程创建、管道、文件描述符、目录遍历以及 Makefile 中用户程序的构建方式。

2.2 实现内容

- `sleep`：解析 tick 数并调用 `sleep` 系统调用；校验缺失参数。
- `pingpong`：建立父子进程间的双向管道，严格关闭不用的文件描述符。
- `primes`：用进程和管道实现并发素数筛，每一级筛选器负责一个素数。
- `find`：递归遍历目录，跳过 `.` 和 `..`，处理 xv6 固定长度目录项名称。
- `xargs`：逐行读取标准输入，将输入项追加到基础参数后执行子进程。
- 可选扩展：实现 `uptime`，暴露系统启动后的 tick 数。

2.3 核心机制

`primes` 展示了 Unix “一个进程做一件事”的组合思想。父筛选器读取当前管道的第一个数作为素数，创建下一级管道，只转发不能被该素数整除的数字。每一级都在写完后关闭写端，使下游能通过 EOF 正常退出，避免进程永久阻塞。

2.4 问题与解决

- 问题：若父子进程保留无关的管道写端，读端永远收不到 EOF。
- 解决：在 `fork` 后立即关闭每个进程不使用的两端，并在转发结束后关闭输出端。
- 问题：xv6 的目录名不是普通的以零结尾字符串。
- 解决：复制 `DIRSIZ` 字节后显式补零，再进行名称比较和路径拼接。

2.5 验证

官方测试覆盖参数错误、系统调用行为、双向通信、素数序列、当前目录与递归查找、`xargs` 参数组合，评分结果见《评分结果》。

3. Lab 2 : System Calls

3.1 实验目的

掌握用户态包装函数、RISC-V `ecall`、系统调用分派表、进程状态和内核数据复制。

3.2 trace 系统调用

为进程控制块增加 `trace` 掩码。`trace(mask)` 设置当前进程的掩码，`fork` 时子进程 继承它。系统调用返回后，分派器依据系统调用号检查对应位，并输出 PID、调用名和 返回值。输出发生在实际调用完成之后，因此得到的返回值是真实结果。

调用链如下：

```
user/trace.c -> user stub -> ecall -> usertrap
               -> syscall() -> sys_trace()/目标系统调用
               -> 条件打印 -> usertrapret
```

3.3 sysinfo 系统调用

`sysinfo` 返回空闲物理内存字节数和活动进程数。内存统计在分配器锁保护下遍历空闲 页链表；进程统计遍历进程表并排除 `UNUSED` 状态。内核先在本地图体中构造结果，再用 `copyout` 写入用户地址，防止直接解引用不可信用户指针。

3.4 问题与解决

- 用户指针可能未映射或越界：统一通过 `argaddr` 和 `copyout` 访问。
- `trace` 必须跨 `fork` 保持：在复制进程状态时同步复制掩码。
- 统计过程存在并发修改：读取共享链表和进程状态时使用对应锁。

4. Lab 3 : Page Tables

4.1 vmprint

递归遍历三级 Sv39 页表，按层级缩进打印有效 PTE。若 PTE 有效且不具有 R/W/X 叶子权限，则它指向下一级页表；否则打印虚拟页到物理页的映射。该工具用于直观理解 根页表、页表页和叶子页之间的关系。

4.2 每进程内核页表

原始 xv6 的所有进程共享一个内核页表。本实验为每个进程创建内核页表，映射 UART、PLIC、virtio、内核代码数据、trampoline，并为该进程的内核栈建立映射。调度到进程 时切换 `satp`，回到调度器时恢复全局内核页表。

生命周期必须对称：

```
allocproc -> 创建内核页表和内核栈映射
scheduler -> 切换 satp + sfence.vma
freeproc  -> 解除映射并释放页表页（不释放设备物理页）
```

4.3 copyin/copyinstr 优化

把用户页表的叶子映射同步到该进程的内核页表，使内核可以直接使用普通内存复制逻辑，减少逐页 `walkaddr` 的开销。扩展、收缩和释放用户地址空间时必须同步更新两套页表，同时保证用户虚拟地址不覆盖内核的 PLIC 区域。

4.4 关键风险

- 释放页表时误释放设备映射指向的物理地址会破坏内核。
- 写 `satp` 后若不执行 `sfence.vma`，TLB 中仍可能保留旧映射。
- 任意一个错误路径漏掉页表页都会形成持续内存泄漏。

5. Lab 4 : Traps

5.1 RISC-V 调用约定

通过汇编练习确认 `a0-a7` 用于参数和返回值、`ra` 保存返回地址、`sp` 指向当前栈，并理解编译器保存寄存器与调用者保存寄存器的区别。

5.2 backtrace

在内核栈中沿 frame pointer (`s0`) 反向遍历栈帧。每个栈帧中，保存的返回地址位于 `fp-8`，上一帧指针位于 `fp-16`。遍历以当前 4 KiB 内核栈页边界为限，避免越界读取。panic 时调用 `backtrace`，可把地址交给 `addr2line` 定位源码。

5.3 alarm 与 sigreturn

进程新增 alarm 周期、已用 tick、处理函数地址、是否正在处理以及 trapframe 备份。时钟中断到来时累加 tick；到期且当前不在 handler 中时，完整保存 trapframe，并把返回用户态的 `epc` 改为处理函数地址。`sigreturn` 恢复备份，使被中断程序像普通函数返回一样继续执行。

防重入标志很关键：若 handler 执行期间再次触发并覆盖备份，第一次中断现场将永久丢失。

6. Lab 5 : Lazy Allocation

6.1 设计

`sbrk(n)` 增长地址空间时只增加 `p->sz`，不立即申请物理页。用户首次访问该地址时，CPU 产生 load/store page fault；内核验证地址后分配清零页、建立用户映射并重试指令。

6.2 边界处理

- fault 地址必须小于进程声明的 `sz`。
- 不允许映射到用户栈 guard page 以下的非法区域。
- 地址按页向下取整，防止同一页重复映射。
- 内存耗尽或 `mappages` 失败时释放临时页并杀死进程。
- 负数 `sbrk` 仍需要立刻解除对应映射。
- `fork`、`copyin` 和 `copyout` 必须容忍合法范围内尚未分配的洞。

6.3 收获

虚拟地址空间大小与实际物理内存占用是两个不同概念。惰性分配把资源成本推迟到真实访问时，代价是缺页路径必须成为正常执行路径，而不能把所有缺页都视为程序错误。

7. Lab 6 : Copy-on-Write Fork

7.1 设计目标

原始 `fork` 复制父进程所有用户页，成本与地址空间大小成正比。COW `fork` 让父子页表 最初共享同一物理页，把可写页改为只读并设置软件 COW 位。任一进程写入时才复制该页。

7.2 引用计数

为每个物理页维护引用计数，并用锁保护：

- `kalloc` 返回的新页计数设为 1。
- COW 共享映射时增加计数。
- `kfree` 先减计数，仅在降到 0 时放回空闲链表。
- 复制失败和页表回滚路径也必须成对减少计数。

7.3 写故障与 `copyout`

用户写 COW 页会产生 `store page fault`。若引用数大于 1，申请新页并复制内容，再把当前页表项改为新物理页和可写权限；若引用数等于 1，可直接恢复写权限。内核的 `copyout` 不会触发用户态缺页，因此也必须显式检测并拆分 COW 页。

7.4 并发与错误路径

引用计数的增减、页表项更新和 TLB 刷新共同决定正确性。尤其要避免两个 CPU 同时释放 最后一个引用，或在 `uvmcopy` 中途失败时遗漏已经增加的引用。

8. Lab 7 : Multithreading

8.1 用户级线程切换

为线程保存 `ra`、`sp` 和 `s0-s11` 等被调用者保存寄存器。创建线程时分配独立栈，把初始 `ra` 指向线程函数；调度器调用汇编 `thread_switch` 保存旧上下文并恢复新上下文。

8.2 并行哈希表

在 `pthread` 程序中为每个 `bucket` 配置互斥锁，使不同桶的 `put/get` 可以并行，同一桶的链表操作仍保持互斥。相比全局锁，这种锁粒度在保证零丢失键的同时提高吞吐量。

8.3 barrier

使用 `mutex`、`condition variable`、到达计数和轮次编号实现可复用屏障。最后到达的线程重置计数、推进轮次并广播；其他线程使用 `while` 等待轮次变化，以抵御虚假唤醒。

9. Lab 8 : Locking

9.1 每 CPU 物理页空闲链表

把单一 `kmem` 空闲链表拆成每 CPU 一条链表，各自有锁。分配优先访问本 CPU 链表；为空时从其他 CPU 偷取一批页。常见分配和释放因此不再竞争同一把全局锁。

中断关闭范围必须覆盖 `cpuid()` 到本地链表操作，否则线程可能迁移到另一 CPU，导致操作错误的链表。

9.2 分桶 `buffer cache`

把块缓存按 `(dev, blockno)` 哈希到多个 `bucket`，每个 `bucket` 独立加锁。查找在目标桶完成；缓存未命中时选择 `refcnt==0` 的最久未使用块并迁移到目标桶。迁移过程需要稳定的加锁顺序或额外淘汰锁，防止死锁和同一磁盘块产生两个缓存副本。

9.3 性能验收

除功能测试外，官方 `kallocstest` 和 `bcachetest` 会统计自旋锁的 `test-and-set` 次数，验证设计确实减少竞争，而不仅是“结果正确”。

10. Lab 9 : File System

10.1 大文件

inode 原本包含 12 个直接块和 1 个一级间接块。实验将一个直接块槽改为双重间接块：

```
inode -> double-indirect block
      -> indirect block
      -> data block
```

`bmap` 按逻辑块号选择直接、一级间接或二级间接路径，并按需分配中间块。`itrunc` 必须 遍历两级索引释放所有数据块和索引块，否则删除大文件后会泄漏磁盘空间。

10.2 符号链接

新增 `symlink(target, path)` 系统调用和 `T_SYMLINK` inode 类型，链接内容保存目标路径。`open` 默认递归解析符号链接；指定 `O_NOFOLLOW` 时返回链接 inode 本身。解析设置最大 深度，阻止自环或链接环导致无限递归。

10.3 事务约束

块分配、inode 更新和目录修改都必须处于日志事务内。错误返回前要正确执行 `iunlockput`、`brelse` 和 `end_op`，否则可能留下锁、引用或未结束事务。

11. Lab 10 : mmap

11.1 VMA 设计

进程维护固定数量的 VMA，记录起始地址、长度、保护位、映射标志、文件偏移和文件引用。
`mmap` 验证参数后选择未用 VMA 和未占用的用户虚拟区，只记录元数据，不立即读文件。

11.2 缺页装入

访问 VMA 内尚未映射的页时，内核：

1. 找到覆盖 `fault` 地址的 VMA。
2. 根据 `PROT_READ/PROT_WRITE` 构造 PTE 权限。
3. 分配并清零物理页。
4. 从文件的对应偏移读取一页。
5. 映射到页对齐后的用户地址。

11.3 `munmap`、`fork` 与 `exit`

`munmap` 支持从头部、尾部或整体解除映射。对 `MAP_SHARED` 且可写的已映射页，在解除前写回文件；未实际 `fault-in` 的页无需处理。`fork` 复制 VMA 元数据并增加文件引用，子进程的物理页仍按需加载；`exit` 自动解除全部 VMA 并关闭引用。

写回文件必须分段执行，避免单次日志事务超过 xv6 日志容量。

12. Lab 11 : Network Driver

12.1 E1000 发送

发送路径读取 TX ring 的下一个描述符，检查 DD 位确认硬件已完成旧请求，释放旧 mbuf，把新 mbuf 的物理地址和长度写入描述符，设置 EOP|RS，最后更新 TDT 通知 网卡。若描述符仍忙则返回失败，不能覆盖硬件正在使用的缓冲区。

12.2 E1000 接收

接收中断处理从 rx_ring 的下一个描述符开始，只处理设置了 DD 的项。把收到的 mbuf 上交网络栈后，立即分配替换 mbuf，重新填写地址并清状态，再更新 RDT 把描述符 归还硬件。环形下标通过取模推进。

12.3 所有权原则

驱动正确性的核心是软件与硬件之间的缓冲区所有权转换：TX 提交后在硬件设置 DD 前不能释放；RX 上交协议栈后原 mbuf 不再属于 ring，必须补充新的缓冲区。

13. 自动化测试与结果

`scripts/grade-all.sh` 使用 `git worktree` 为每个实验创建临时工作树，避免切换当前分支 或 让不同 Lab 的构建产物互相污染。每项测试最长 900 秒，完整日志写入 `artifacts/grades/`。执行方式：

```
./scripts/grade-all.sh
```

正式测试结果与运行日期记录在 [评分结果（评分结果.md）](#)。11 个 Lab 均取得官方满分。评分器不仅检查输出，还会执行 `usertests`、压力测试、并发竞争统计和异常参数测试。

14. 项目难点总结

1. **跨模块不变量**：COW 同时涉及页表权限、分配器引用计数、缺页和 `copyout`，任何单点实现都不足以保证正确。
2. **资源生命周期**：页表页、文件引用、inode、buffer、mbuf 和锁都要求成功与失败路径严格对称。
3. **并发正确性与性能**：锁实验不仅要求无竞争错误，还要求降低锁冲突；锁粒度、顺序和 CPU 迁移必须一起考虑。
4. **惰性机制**：lazy allocation 和 mmap 把工作推迟到 fault 路径，所有传统上假设“页面必然存在”的代码都需要重新审视。
5. **工具链演进**：旧课程代码在新版 GCC/binutils/QEMU 上存在兼容问题，需要区分真正的代码缺陷与工具链诊断差异。

15. 实验心得

完成全部实验后，对操作系统的理解从“模块功能”转变为“状态与不变量”。系统调用看似只是增加一个编号，但必须贯穿用户 stub、陷阱、参数复制、权限和进程生命周期；一次页错误也不只是分配内存，而是页表、TLB、物理页所有权和错误恢复的联合操作。

xv6 代码量不大，因此可以沿着一次调用完整追踪到硬件边界。与此同时，COW、buffer cache 和网卡 ring 表明，即使在教学内核中，并发和资源所有权仍是最容易产生严重错误的部分。官方评分器的价值在于把正常路径、边界条件、压力与性能约束统一纳入验收。

16. 参考资料

1. MIT 6.S081 Operating System Engineering, Fall 2020.
2. Russ Cox, Frans Kaashoek, Robert Morris, xv6: a simple, Unix-like teaching operating system.
3. The RISC-V Instruction Set Manual.
4. Intel 8254x Family of Gigabit Ethernet Controllers Software Developer's Manual.
5. 项目仓库中的课程要求 PDF 与 xv6 Labs 项目说明。